

4	addl	%r8d, %eax	Add to result
5	addq	\$1, %rdx	j++
6	addq	%r9, %rcx	Bptr += n
7	cmpq	%rdi, %rdx	Compare j:n
8	jne	.L24	If !=, goto loop

我们看到程序既使用了伸缩过的值 $4n$ (寄存器 `%r9`) 来增加 `Bptr`, 也使用了 n 的值 (寄存器 `%rdi`) 来检查循环的边界。C 代码中并没有体现出需要这两个值, 但是由于指针运算的伸缩, 才使用了这两个值。

可以看到, 如果允许使用优化, GCC 能够识别出程序访问多维数组的元素的步长。然后生成的代码会避免直接应用等式 (3.1) 会导致的乘法。不论生成基于指针的代码 (图 3-37b) 还是基于数组的代码 (图 3-38b), 这些优化都能显著提高程序的性能。

3.9 异质的数据结构

C 语言提供了两种将不同类型的对象组合到一起创建数据类型的机制: 结构 (structure), 用关键字 `struct` 来声明, 将多个对象集合到一个单位中; 联合 (union), 用关键字 `union` 来声明, 允许用几种不同的类型来引用一个对象。

3.9.1 结构

C 语言的 `struct` 声明创建一个数据类型, 将可能不同类型的对象聚合到一个对象中。用名字来引用结构的各个组成部分。类似于数组的实现, 结构的所有组成部分都存放在内存中一段连续的区域内, 而指向结构的指针就是结构第一个字节的地址。编译器维护关于每个结构类型的信息, 指示每个字段 (field) 的字节偏移。它以这些偏移作为内存引用指令中的位移, 从而产生对结构元素的引用。

给 C 语言初学者 将一个对象表示为 `struct`

C 语言提供的 `struct` 数据类型的构造函数 (constructor) 与 C++ 和 Java 的对象最为接近。它允许程序员在一个数据结构中保存关于某个实体的信息, 并用名字来引用这些信息。

例如, 一个图形程序可能要用结构来表示一个长方形:

```
struct rect {
    long llx;           /* X coordinate of lower-left corner */
    long lly;           /* Y coordinate of lower-left corner */
    unsigned long width; /* Width (in pixels) */
    unsigned long height; /* Height (in pixels) */
    unsigned color;      /* Coding of color */
};
```

可以声明一个 `struct rect` 类型的变量 `r`, 并将它的字段值设置如下:

```
struct rect r;
r.llx = r.lly = 0;
r.color = 0xFF00FF;
r.width = 10;
r.height = 20;
```

这里表达式 `r.llx` 就会选择结构 `r` 的 `llx` 字段。

另外, 我们可以在一条语句中既声明变量又初始化它的字段:

```
struct rect r = { 0, 0, 10, 20, 0xFF00FF};
```

将指向结构的指针从一个地方传递到另一个地方，而不是复制它们，这是很常见的。例如，下面的函数计算长方形的面积，这里，传递给函数的就是一个指向长方形 struct 的指针：

```
long area(struct rect *rp) {
    return (*rp).width * (*rp).height;
}
```

表达式 `(*rp).width` 间接引用了这个指针，并且选取所得结构的 `width` 字段。这里必须要用括号，因为编译器会将表达式 `*rp.width` 解释为 `*(rp.width)`，而这是非法的。间接引用和字段选取结合起来使用非常常见，以至于 C 语言提供了一种替代的表示法 `->`。即 `rp-> width` 等价于表达式 `(*rp).width`。例如，我们可以写一个函数，它将一个长方形顺时针旋转 90 度：

```
void rotate_left(struct rect *rp) {
    /* Exchange width and height */
    long t = rp->height;
    rp->height = rp->width;
    rp->width = t;
    /* Shift to new lower-left corner */
    rp->llx -= t;
}
```

C++ 和 Java 的对象比 C 语言中的结构要复杂精细得多，因为它们将一组可以被调用来执行计算的方法与一个对象联系起来。在 C 语言中，我们可以简单地把这些方法写成普通函数，就像上面所示的函数 `area` 和 `rotate_left`。

让我们来看看这样一个例子，考虑下面这样的结构声明：

```
struct rec {
    int i;
    int j;
    int a[2];
    int *p;
};
```

这个结构包括 4 个字段：两个 4 字节 `int`、一个由两个类型为 `int` 的元素组成的数组和一个 8 字节整型指针，总共是 24 个字节：

偏移	0	4	8	16	24
内容	i	j	a[0]	a[1]	p

可以观察到，数组 `a` 是嵌入到这个结构中的。上图中顶部的数字给出的是各个字段相对于结构开始处的字节偏移。

为了访问结构的字段，编译器产生的代码要将结构的地址加上适当的偏移。例如，假设 `struct rec*` 类型的变量 `r` 放在寄存器 `%rdi` 中。那么下面的代码将元素 `r->i` 复制到元素 `r->j`：

```
Registers: r in %rdi
1    movl    (%rdi), %eax          Get r->i
2    movl    %eax, 4(%rdi)        Store in r->j
```


因为字段 i 的偏移量为 0，所以这个字段的地址就是 r 的值。为了存储到字段 j ，代码要将 r 的地址加上偏移量 4。

要产生一个指向结构内部对象的指针，我们只需将结构的地址加上该字段的偏移量。例如，只用加上偏移量 $8+4\times 1=12$ ，就可以得到指针 $\&(r\rightarrow a[1])$ 。对于在寄存器 $\%rdi$ 中的指针 r 和在寄存器 $\%rsi$ 中的长整数变量 i ，我们可以用一条指令产生指针 $\&(r\rightarrow a[i])$ 的值：

```
Registers: r in %rdi, i %rsi
1    leaq    8(%rdi,%rsi,4), %rax    Set %rax to &r->a[i]
```

最后举一个例子，下面的代码实现的是语句：

```
r->p = &r->a[r->i + r->j];
```

开始时 r 在寄存器 $\%rdi$ 中：

```
Registers: r in %rdi
1    movl    4(%rdi), %eax           Get r->j
2    addl    (%rdi), %eax           Add r->i
3    cltq                               Extend to 8 bytes
4    leaq    8(%rdi,%rax,4), %rax    Compute &r->a[r->i + r->j]
5    movq    %rax, 16(%rdi)         Store in r->p
```

综上所述，结构的各个字段的选取完全是在编译时处理的。机器代码不包含关于字段声明或字段名字的信息。

 **练习题 3.41** 考虑下面的结构声明：

```
struct prob {
    int *p;
    struct {
        int x;
        int y;
    } s;
    struct prob *next;
};
```

这个声明说明一个结构可以嵌套在另一个结构中，就像数组可以嵌套在结构中、数组可以嵌套在数组中一样。

下面的过程(省略了某些表达式)对这个结构进行操作：

```
void sp_init(struct prob *sp) {
    sp->s.x = _____;
    sp->p   = _____;
    sp->next = _____;
}
```

A. 下列字段的偏移量是多少(以字节为单位)?

```
p:      _____
s.x:    _____
s.y:    _____
next:   _____
```

B. 这个结构总共需要多少字节?

C. 编译器为 `sp_init` 的主体产生的汇编代码如下：

```

void sp_init(struct prob *sp)
    sp in %rdi
1   sp_init:
2       movl    12(%rdi), %eax
3       movl    %eax, 8(%rdi)
4       leaq    8(%rdi), %rax
5       movq    %rax, (%rdi)
6       movq    %rdi, 16(%rdi)
7       ret

```

根据这些信息，填写 `sp_init` 代码中缺失的表达式。



练习题 3.42 下面的代码给出了类型 `ELE` 的结构声明以及函数 `fun` 的原型：

```

struct ELE {
    long    v;
    struct ELE *p;
};

```

```

long fun(struct ELE *ptr);

```

当编译 `fun` 的代码时，GCC 会产生如下汇编代码：

```

long fun(struct ELE *ptr)
    ptr in %rdi
1   fun:
2       movl    $0, %eax
3       jmp     .L2
4   .L3:
5       addq    (%rdi), %rax
6       movq    8(%rdi), %rdi
7   .L2:
8       testq   %rdi, %rdi
9       jne     .L3
10      rep; ret

```

- A. 利用逆向工程技巧写出 `fun` 的 C 代码。
- B. 描述这个结构实现的数据结构以及 `fun` 执行的操作。

3.9.2 联合

联合提供了一种方式，能够规避 C 语言的类型系统，允许以多种类型来引用一个对象。联合声明的语法与结构的语法一样，只不过语义相差比较大。它们是用不同的字段来引用相同的内存块。

考虑下面的声明：

```

struct S3 {
    char c;
    int i[2];
    double v;
};
union U3 {
    char c;
    int i[2];
    double v;
};

```

在一台 x86-64 Linux 机器上编译时,字段的偏移量、数据类型 S3 和 U3 的完整大小如下:

类型	c	i	v	大小
S3	0	4	16	24
U3	0	0	0	8

(稍后会解释 S3 中 i 的偏移量为什么是 4 而不是 1,以及为什么 v 的偏移量是 16 而不是 9 或 12。)对于类型 union U3 * 的指针 p, p-> c、p-> i[0] 和 p-> v 引用的都是数据结构的起始位置。还可以观察到,一个联合的总的大小等于它最大字段的大小。

在一些下上文中,联合十分有用。但是,它也能引起一些讨厌的错误,因为它们绕过了 C 语言类型系统提供的安全措施。一种应用情况是,我们事先知道对一个数据结构中的两个不同字段的使用是互斥的,那么将这两个字段声明为联合的一部分,而不是结构的一部分,会减小分配空间的总量。

例如,假设我们想实现一个二叉树的数据结构,每个叶子节点都有两个 double 类型的数据值,而每个内部节点都有指向两个孩子节点的指针,但是没有数据。如果声明如下:

```
struct node_s {
    struct node_s *left;
    struct node_s *right;
    double data[2];
};
```

那么每个节点需要 32 个字节,每种类型的节点都要浪费一半的字节。相反,如果我们如下声明一个节点:

```
union node_u {
    struct {
        union node_u *left;
        union node_u *right;
    } internal;
    double data[2];
};
```

那么,每个节点就只需要 16 个字节。如果 n 是一个指针,指向 union node_u * 类型的节点,我们用 n-> data[0] 和 n-> data[1] 来引用叶子节点的数据,而用 n-> internal.left 和 n-> internal.right 来引用内部节点的孩子。

不过,如果这样编码,就没有办法来确定一个给定的节点到底是叶子节点,还是内部节点。通常的方法是引入一个枚举类型,定义这个联合中可能的不同选择,然后再创建一个结构,包含一个标签字段和这个联合:

```
typedef enum { N_LEAF, N_INTERNAL } nodetype_t;

struct node_t {
    nodetype_t type;
    union {
        struct {
            struct node_t *left;
            struct node_t *right;
        } internal;
        double data[2];
    } info;
};
```


这个结构总共需要 24 个字节：type 是 4 个字节，info.internal.left 和 info.internal.right 各要 8 个字节，或者是 info.data 要 16 个字节。我们后面很快会谈到，在字段 type 和联合的元素之间需要 4 个字节的填充，所以整个结构大小为 $4+4+16=24$ 。在这种情况下，相对于给代码造成的麻烦，使用联合带来的节省是很小的。对于有较多字段的数据结构，这样的节省会更加吸引人。

联合还可以用来访问不同数据类型的位模式。例如，假设我们使用简单的强制类型转换将一个 double 类型的值 d 转换为 unsigned long 类型的值 u：

```
unsigned long u = (unsigned long) d;
```

值 u 会是 d 的整数表示。除了 d 的值为 0.0 的情况以外，u 的位表示会与 d 的很不一样。再看下面这段代码，从一个 double 产生一个 unsigned long 类型的值：

```
unsigned long double2bits(double d) {
    union {
        double d;
        unsigned long u;
    } temp;
    temp.d = d;
    return temp.u;
};
```

在这段代码中，我们以一种数据类型来存储联合中的参数，又以另一种数据类型来访问它。结果会是 u 具有和 d 一样的位表示，包括符号位字段、指数和尾数，如 3.11 节中描述的那样。u 的数值与 d 的数值没有任何关系，除了 d 等于 0.0 的情况。

当用联合来将各种不同大小的数据类型结合到一起时，字节顺序问题就变得很重要了。例如，假设我们写了一个过程，它以两个 4 字节的 unsigned 的位模式，创建一个 8 字节的 double：

```
double uu2double(unsigned word0, unsigned word1)
{
    union {
        double d;
        unsigned u[2];
    } temp;

    temp.u[0] = word0;
    temp.u[1] = word1;
    return temp.d;
}
```

在 x86-64 这样的小端法机器上，参数 word0 是 d 的低位 4 个字节，而 word1 是高位 4 个字节。在大端法机器上，这两个参数的角色刚好相反。

 **练习题 3.43** 假设给你个任务，检查一下 C 编译器为结构和联合的访问产生正确的代码。你写了下面的结构声明：

```
typedef union {
    struct {
        long    u;
        short   v;
        char     w;
```

```
    } t1;
    struct {
        int a[2];
        char *p;
    } t2;
} u_type;
```

你写了一组具有下面这种形式的函数：

```
void get(u_type *up, type *dest) {
    *dest = expr;
}
```

这组函数有不一样的访问表达式 *expr*，而且根据 *expr* 的类型来设置目的数据类型 *type*。然后再检查编译这些函数时产生的代码，看看它们是否与你预期的一样。

假设在这些函数中，*up* 和 *dest* 分别被加载到寄存器 *%rdi* 和 *%rsi* 中。填写下表中的数据类型 *type*，并用 1~3 条指令序列来计算表达式，并将结果存储到 *dest* 中。

<i>expr</i>	<i>type</i>	代码
<i>up</i> -> <i>t1.u</i>	long	movq(<i>%rdi</i>), <i>%rax</i> movq <i>%rax</i> , (<i>%rsi</i>)
<i>up</i> -> <i>t1.v</i>		
& <i>up</i> -> <i>t1.w</i>		
<i>up</i> -> <i>t2.a</i>		
<i>up</i> -> <i>t2.a</i> [<i>up</i> -> <i>t1.u</i>]		
* <i>up</i> -> <i>t2.p</i>		

3.9.3 数据对齐

许多计算机系统对基本数据类型的合法地址做出了一些限制，要求某种类型对象的地址必须是某个值 *K* (通常是 2、4 或 8) 的倍数。这种对齐限制简化了形成处理器和内存系统之间接口的硬件设计。例如，假设一个处理器总是从内存中取 8 个字节，则地址必须为 8 的倍数。如果我们能保证将所有的 *double* 类型数据的地址对齐成 8 的倍数，那么就可以用一个内存操作来读或者写值了。否则，我们可能需要执行两次内存访问，因为对象可能被分放在两个 8 字节内存块中。

无论数据是否对齐，x86-64 硬件都能正确工作。不过，Intel 还是建议要对齐数据以提高内存系统的性能。对齐原则是任何 *K* 字节的基本对象的地址必须是 *K* 的倍数。可以看到这条原则会得到如下对齐：

K	类型
1	char
2	short
4	int, float
8	long, double, char*

确保每种数据类型都是按照指定方式来组织和分配，即每种类型的对象都满足它的对齐限制，就可保证实施对齐。编译器在汇编代码中放入命令，指明全局数据所需的对齐。例如，3.6.8 节开始的跳转表的汇编代码声明在第 2 行包含下面这样的命令：

```
.align 8
```

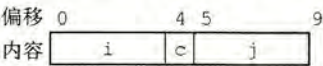
这就保证了它后面的数据(在此，是跳转表的开始)的起始地址是 8 的倍数。因为每个表项长 8 个字节，后面的元素都会遵守 8 字节对齐的限制。

对于包含结构的代码，编译器可能需要在字段的分配中插入间隙，以保证每个结构元素都满足它的对齐要求。而结构本身对它的起始地址也有一些对齐要求。

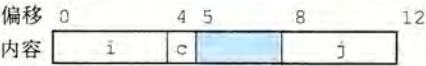
比如说，考虑下面的结构声明：

```
struct S1 {
    int i;
    char c;
    int j;
};
```

假设编译器用最小的 9 字节分配，画出图来是这样的：



它是不可能满足字段 i(偏移为 0)和 j(偏移为 5)的 4 字节对齐要求的。取而代之地，编译器在字段 c 和 j 之间插入一个 3 字节的间隙(在此用蓝色阴影表示)：



结果，j 的偏移量为 8，而整个结构的大小为 12 字节。此外，编译器必须保证任何 struct S1 * 类型的指针 p 都满足 4 字节对齐。用我们前面的符号，设指针 p 的值为 x_p 。那么， x_p 必须是 4 的倍数。这就保证了 $p \rightarrow i$ (地址 x_p)和 $p \rightarrow j$ (地址 $x_p + 8$)都满足它们的 4 字节对齐要求。

另外，编译器结构的末尾可能需要一些填充，这样结构数组中的每个元素都会满足它的对齐要求。例如，考虑下面这个结构声明：

```
struct S2 {
    int i;
    int j;
    char c;
};
```

如果我们将这个结构打包成 9 个字节，只要保证结构的起始地址满足 4 字节对齐要求，我们仍然能够保证满足字段 i 和 j 的对齐要求。不过，考虑下面的声明：

```
struct S2 d[4];
```


分配9个字节,不可能满足d的每个元素的对齐要求,因为这些元素的地址分别为 x_d 、 x_d+9 、 x_d+18 和 x_d+27 。相反,编译器会为结构s2分配12个字节,最后3个字节是浪费的空间:

偏移	0	4	8	9	12
内容	i	j	c		

这样一来,d的元素地址分别为 x_d 、 x_d+12 、 x_d+24 和 x_d+36 。只要 x_d 是4的倍数,所有的对齐限制就都可以满足了。

 **练习题 3.44** 对下面每个结构声明,确定每个字段的偏移量、结构总的大小,以及在x86-64下它的对齐要求:

- struct P1 { int i; char c; int j; char d; };
- struct P2 { int i; char c; char d; long j; };
- struct P3 { short w[3]; char c[3] };
- struct P4 { short w[5]; char *c[3] };
- struct P5 { struct P3 a[2]; struct P2 t };

 **练习题 3.45** 对于下列结构声明回答后续问题:

```
struct {
    char    *a;
    short   b;
    double  c;
    char    d;
    float   e;
    char    f;
    long    g;
    int     h;
} rec;
```

- 这个结构中所有的字段的字节偏移量是多少?
- 这个结构总的大小是多少?
- 重新排列这个结构中的字段,以最小化浪费的空间,然后再给出重排过的结构的字节偏移量和总的大小。

旁注 强制对齐的情况

对于大多数x86-64指令来说,保持数据对齐能够提高效率,但是它不会影响程序的行为。另一方面,如果数据没有对齐,某些型号的Intel和AMD处理器对于有些实现多媒体操作的SSE指令,就无法正确执行。这些指令对16字节数据块进行操作,在SSE单元和内存之间传送数据的指令要求内存地址必须是16的倍数。任何试图以不满足对齐要求的地址来访问内存都会导致异常(参见8.1节),默认的行为是程序终止。

因此,任何针对x86-64处理器的编译器和运行时系统都必须保证分配用来保存可能会被SSE寄存器读或写的数据结构的内存,都必须满足16字节对齐。这个要求有两个后果:

- 任何内存分配函数(alloca、malloc、calloc或realloc)生成的块的起始地址都必须是16的倍数。
- 大多数函数的栈帧的边界都必须是16字节的倍数。(这个要求有一些例外。)

较近版本的x86-64处理器实现了AVX多媒体指令。除了提供SSE指令的超集,支持AVX的指令并没有强制性的对齐要求。

3.10 在机器级程序中将控制与数据结合起来

到目前为止，我们已经分别讨论机器级代码如何实现程序的控制部分和如何实现不同的数据结构。在本节中，我们会看看数据和控制如何交互。首先，深入审视一下指针，它是 C 编程语言中最重要的概念之一，但是许多程序员对它的理解都非常浅显。我们复习符号调试器 GDB 的使用，用它仔细检查机器级程序的详细运行。接下来，看看理解机器级程序如何帮助我们研究缓冲区溢出，这是现实世界许多系统中一种很重要的安全漏洞。最后，查看机器级程序如何实现函数要求的栈空间大小在每次执行时都可能不同的情况。

3.10.1 理解指针

指针是 C 语言的一个核心特色。它们以一种统一方式，对不同数据结构中的元素产生引用。对于编程新手来说，指针总是会带来很多的困惑，但是基本概念其实非常简单。在此，我们重点介绍一些指针和它们映射到机器代码的关键原则。

- 每个指针都对应一个类型。这个类型表明该指针指向的是哪一类对象。以下面的指针声明为例：

```
int *ip;
char **cpp;
```

变量 `ip` 是一个指向 `int` 类型对象的指针，而 `cpp` 指针指向的对象自身就是一个指向 `char` 类型对象的指针。通常，如果对象类型为 `T`，那么指针的类型为 `T*`。特殊的 `void *` 类型代表通用指针。比如说，`malloc` 函数返回一个通用指针，然后通过显式强制类型转换或者赋值操作那样的隐式强制类型转换，将它转换成一个有类型的指针。指针类型不是机器代码中的一部分；它们是 C 语言提供的一种抽象，帮助程序员避免寻址错误。

- 每个指针都有一个值。这个值是某个指定类型的对象的地址。特殊的 `NULL(0)` 值表示该指针没有指向任何地方。
- 指针用 `&` 运算符创建。这个运算符可以应用到任何 `lvalue` 类的 C 表达式上，`lvalue` 意指可以出现在赋值语句左边的表达式。这样的例子包括变量以及结构、联合和数组的元素。我们已经看到，因为 `leaq` 指令是设计用来计算内存引用的地址的，`&` 运算符的机器代码实现常常用这条指令来计算表达式的值。
- `*` 操作符用于间接引用指针。其结果是一个值，它的类型与该指针的类型一致。间接引用是用内存引用来实现的，要么是存储到一个指定的地址，要么是从指定的地址读取。
- 数组与指针紧密联系。一个数组的名字可以像一个指针变量一样引用（但是不能修改）。数组引用（例如 `a[3]`）与指针运算和间接引用（例如 `*(a+3)`）有一样的效果。数组引用和指针运算都需要用对象大小对偏移量进行伸缩。当我们写表达式 `p+i`，这里指针 `p` 的值为 `p`，得到的地址计算为 `p+L·i`，这里 `L` 是与 `p` 相关联的数据类型的大小。
- 将指针从一种类型强制转换成另一种类型，只改变它的类型，而不改变它的值。强制类型转换的一个效果是改变指针运算的伸缩。例如，如果 `p` 是一个 `char *` 类型的指针，它的值为 `p`，那么表达式 `(int *)p+7` 计算为 `p+28`，而 `(int *) (p+7)` 计算为 `p+7`。（回想一下，强制类型转换的优先级高于加法。）